

Learnspace — Продуктовая и техническая спецификация

Учебный workspace для целенаправленного изучения тем без отвлечений.

Этот документ — единственный источник истины по продукту, стеку и плану реализации. К нему возвращаешься при любом сомнении и решении.

Содержание

1. [Продукт](#)
 2. [Скоуп MVP](#)
 3. [Технический стек](#)
 4. [Модель данных и API](#)
 5. [Дизайн-направление](#)
 6. [План реализации](#)
 7. [Риски и принципы работы](#)
-

1. Продукт

Одна фраза

Учебный workspace для студентов 18-25 лет, которые хотят глубоко разобраться в конкретной теме за фокусную сессию (1-3+ часа) — без рекомендаций, лент и алгоритмических отвлечений.

Главное обещание

Продукт не борется за внимание пользователя — он его защищает.

Это не «ещё один агрегатор» и не «AI-помощник для обучения». Это место, куда приходят с намерением изучить что-то, и где архитектура продукта защищает это намерение от сноса в YouTube-кроличьи норы и информационный шум.

Главный пользователь

Студент университета 18-25 лет, который разбирается в конкретной теме по своему предмету. Сценарий — фокусная сессия 30 минут до нескольких часов на одну тему («понять трансформеры к завтрашнему семинару», «разобраться с байесовской статистикой»). Не марафонская подготовка к ЕГЭ, не школьник 7 класса, не профессионал, осваивающий новую профессию.

Ключевая механика

При регистрации пользователь задаёт **дефолты профиля**: уровень (новичок / средний / продвинутый), типичная длительность сессии, баланс видео/текста, актуальность контента по умолчанию.

При создании каждого нового workspace пользователь:

1. Указывает **тему/цель** в свободной форме (обязательно)
2. При желании раскрывает «Дополнительно ▾» и подстраивает фильтры под этот конкретный случай (опционально — иначе используются дефолты профиля)

Приложение использует эти параметры для подбора материалов из открытых источников (YouTube, веб через Brave Search). **Пользователь сам выбирает**, что добавить в workspace из подборки. Алгоритм только фильтрует, не ранжирует «лучшее».

Затем пользователь смотрит и читает прямо в приложении — без рекомендаций, лент и автоплея — и сохраняет полезное с заметками. Возвращается к workspace, продолжает изучение. Сам решает, когда подгрузить новые материалы.

Что отличает от существующих решений

1. **Курирование под конкретную цель.** YouTube без рекомендаций — это «YouTube без рекомендаций», но всё ещё требует самостоятельного поиска. Здесь приложение делает работу подбора.
 2. **Принудительный фокус через архитектуру продукта.** Нет «похожих видео», нет автоплея, нет ленты. Только то, что пользователь сам положил в workspace.
 3. **Явный контракт пользователя с приложением.** YouTube никогда не спросит «сколько ты планируешь учиться, какая твоя цель». Здесь пользователь *декларирует намерение*, и продукт работает с ним явно.
-

2. Скоуп MVP

Must have — без этого нет продукта

1. **Регистрация и аутентификация.** Email + пароль. OAuth (Google) опционально.
2. **Onboarding профиля при первой регистрации.** 4-5 экранов: о продукте → уровень → длительность → баланс источников → актуальность. Можно пропустить.
3. **BYO API keys.** Минимум: YouTube Data API + Brave Search обязательно (без них продукт не работает). Anthropic API — опционально, для Should-фич. Зашифрованное хранение в БД (AES-256-GCM).
4. **Создание workspace.** Тема/цель в свободной форме + опциональная подстройка фильтров. Сразу запускается поиск.
5. **Поиск и подборка.** Параллельные запросы к YouTube + Brave Search с применёнными фильтрами. Карточки результатов: превью, заголовок, источник, длительность, дата. Без своего ранжирования (порядок от источников).
6. **Просмотр видео внутри приложения.** YouTube IFrame с параметрами `rel=0` , `modestbranding=1` , `iv_load_policy=3` , `enablejsapi=1` . Перехват события `ended` → `stopVideo()` для скрытия сетки рекомендаций. Свой UI поверх плеера для перехода к следующему ресурсу.
7. **Reader-view для статей.** Mozilla Readability на backend → чистый HTML → рендер с шрифтом Source Serif. Fallback «Открыть оригинал» при ошибке парсинга. Контент статей не сохраняется в БД (копирайт).
8. **Сохранение в workspace.** Кнопка «Добавить в workspace» на карточках и в просмотре.
9. **Markdown-заметки.** TipTap-редактор, привязанный к ресурсу. Для видео — кнопка «Вставить таймкод» вставляет текущее время плеера. Auto-save с debounce.
10. **Список workspace'ов.** Главный экран после логина. Прогресс по каждому (например, «3 из 7 ресурсов просмотрено»).
11. **Workspace view.** Цель сверху, сохранённые ресурсы списком, заметки сбоку. Кнопка «Обновить подборку».
12. **Удаление и редактирование.** Workspace, ресурс, заметка — soft delete с

восстановлением в течение 30 дней.

13. **Базовые настройки.** Изменение дефолтов профиля, управление API-ключами, тема (светлая / тёмная / system), выход, удаление аккаунта.

14. **Светлая и тёмная темы.** Обе первого класса.

Should have — после стабильного MVP

- AI-резюме материала через Claude (Anthropic API пользователя)
- Поиск по сохранённому контенту
- Экспорт workspace в markdown
- Улучшенная визуализация прогресса просмотра

Won't have for MVP — явно откладываем

- Learning plans / структурированные курсы
- Социальные фичи / шеринг workspace
- Флешкарты / spaced repetition
- AI-агент, ведущий через материал
- Книги, подкасты, академические PDF как источники
- Мобильное приложение (вторая фаза)
- Командная работа / организации
- Платные тарифы и биллинг (модель — open-source + BYO keys)
- Аналитика обучения / стрики / геймификация
- Коллекции / папки внутри workspace
- Inbox для быстрых сохранений без workspace
- Интеграции с Notion / Obsidian (экспорт в markdown покрывает 80% случаев)
- Push-уведомления (кроме явно запрошенных пользователем напоминаний)

Реалистичные ограничения, которые надо принять

- **Реклама на монетизированных видео YouTube.** Не отключается через IFrame API. Решение — честное предупреждение в UI и предпочтение академических каналов в подборке. Не обещать «полностью без рекламы» в

маркетинге.

- **Логотип YouTube в плеере.** Маленькая иконка остаётся даже при `modestbranding=1`. Это требование ToS.
 - **Парсинг произвольных веб-страниц несовершенен.** Mozilla Readability работает на ~80% сайтов. Для остальных — fallback «Открыть оригинал».
-

3. Технический стек

Frontend (web)

- **Next.js 15** (App Router) — серверные компоненты, SSR для SEO landing/docs
- **TypeScript** — обязателен везде
- **Tailwind CSS + shadcn/ui** — UI-компоненты копируются в проект, можно кастомизировать
- **TanStack Query** — серверное состояние, кэш
- **Zustand** — локальное UI-состояние
- **TipTap** — markdown-редактор для заметок
- **react-youtube** или прямая интеграция YouTube IFrame API — видео-плеер

Backend

- **Hono** (TypeScript) — фреймворк, ложится на Railway, перспективнее Express
- **tRPC** — типобезопасные вызовы между фронтом и бэком, автокомплит из коробки
- **Drizzle ORM** — типобезопасный SQL-билдер, лучше Prisma для перформанса
- **Postgres** (managed на Railway)
- **Better Auth** — аутентификация без вендор-лока, работает с Drizzle
- **@mozilla/readability** — парсинг статей в reader-view

Внешние сервисы (BYO keys)

- **YouTube Data API v3** — поиск видео, метаданные. Бесплатно, 10k units/день у пользователя.

- **Brave Search API** — веб-поиск. \$3/1000 запросов, 2000 free/мес. Лучшая поддержка многих языков, включая русский.
- **Anthropic API** — для Should-фич (AI-резюме). Опционально, не обязательно для базового использования.

Инфраструктура

- **Railway** — backend (Hono в Docker) + Postgres
- **Vercel** — frontend (Next.js, бесплатно для open-source)
- **Cloudflare** — DNS + защита от DDoS
- **GitHub** — репозиторий + Actions для CI

Менеджер пакетов и оркестрация

- **pnpm** — быстрее npm/yarn, лучше работает с монорепо
- **Turborepo** — кэширование сборок, параллелизация задач

Структура монорепо

```

learnspace/
├─ apps/
│  └─ web/                # Next.js (landing + docs + app)
│     └─ api/             # Hono backend
├─ packages/
│  └─ db/                 # Drizzle схема и миграции
│     └─ api-types/       # Общие типы (tRPC роутеры)
│        └─ core/         # Бизнес-логика (поиск, фильтрация, парсинг)
│           └─ ui/         # Общие UI-компоненты (shadcn/ui)
│              └─ config/  # ESLint, TypeScript, Tailwind конфиги
├─ package.json
├─ pnpm-workspace.yaml
├─ turbo.json
└─ README.md

```

Шифрование API-ключей

- AES-256-GCM на стороне backend
- Master key — переменная окружения на Railway (`ENCRYPTION_KEY`)
- При запросе backend расшифровывает ключ пользователя и проксирует запрос к провайдеру (YouTube/Brave/Anthropic)

- Anthropic-вызовы могут идти и напрямую с клиента через `dangerouslyAllowBrowser` + `anthropic-dangerous-direct-browser-access` header — это снижает нагрузку на backend и не требует расшифровки на сервере. Решение по каждому конкретному вызову — на этапе реализации.
-

4. Модель данных и API

Принципы

1. **Не хранить контент, который нам не принадлежит.** Только URL, метаданные, заметки пользователя.
2. **Гибкие JSON-поля для эволюции.** `metadata_json`, `settings_json`, `filters_json` — чтобы добавлять новое без миграций.
3. **Soft delete для workspace, resources, notes** с физическим удалением через 30 дней (cron-job).

Схема БД

users

<code>id</code>	<code>uuid pk</code>
<code>email</code>	<code>text unique</code>
<code>email_verified</code>	<code>bool</code>
<code>password_hash</code>	<code>text</code>
<code>display_name</code>	<code>text</code>
<code>created_at</code>	<code>timestamp</code>
<code>updated_at</code>	<code>timestamp</code>

user_settings

<code>user_id</code>	<code>uuid pk fk → users.id</code>
<code>default_level</code>	<code>text ('beginner' 'intermediate' 'advanced')</code>
<code>default_duration</code>	<code>text ('short' 'medium' 'long')</code>
<code>default_balance</code>	<code>text ('video' 'text' 'mixed')</code>
<code>default_freshness</code>	<code>text ('any' '6m' '1y' '2y')</code>
<code>ui_theme</code>	<code>text ('light' 'dark' 'system')</code>
<code>ui_language</code>	<code>text ('en' 'ru' ...)</code>
<code>onboarding_completed</code>	<code>bool</code>
<code>created_at</code>	<code>timestamp</code>

updated_at	timestamp
------------	-----------

user_api_keys

id	uuid pk
user_id	uuid fk → users.id
provider	text ('anthropic' 'youtube' 'brave')
encrypted_key	text (AES-256-GCM)
key_iv	text (initialization vector)
last_used_at	timestamp nullable
last_validated_at	timestamp nullable
is_valid	bool
created_at	timestamp
unique(user_id, provider)	

workspaces

id	uuid pk
user_id	uuid fk → users.id
title	text
goal	text
filters_json	jsonb (level, duration, balance, freshness)
metadata_json	jsonb
created_at	timestamp
updated_at	timestamp
last_opened_at	timestamp
deleted_at	timestamp nullable

resources

id	uuid pk
workspace_id	uuid fk → workspaces.id
url	text
type	text ('video' 'article')
source	text ('youtube' 'web')
title	text
description	text nullable
thumbnail_url	text nullable
duration_seconds	int nullable
published_at	timestamp nullable
metadata_json	jsonb
saved_at	timestamp
last_opened_at	timestamp nullable

progress_seconds	int	default 0
is_completed	bool	default false
deleted_at	timestamp	nullable

notes

id	uuid	pk
resource_id	uuid	fk → resources.id
content_md	text	
timestamp_seconds	int	nullable
created_at	timestamp	
updated_at	timestamp	
deleted_at	timestamp	nullable

resource_summaries

id	uuid	pk
resource_id	uuid	fk → resources.id
content_md	text	
summary_type	text	('short' 'detailed')
model_used	text	
tokens_used	int	
created_at	timestamp	

(Закладывается заранее, используется в Should-фазе.)

search_queries

id	uuid	pk
workspace_id	uuid	fk → workspaces.id
query_text	text	
filters_json	jsonb	
results_count	int	
created_at	timestamp	

search_results_cache

id	uuid	pk
query_hash	text	indexed (sha256 or query+filters+provider)
provider	text	
results_json	jsonb	
created_at	timestamp	

```
expires_at    timestamp
```

(TTL 24 часа.)

API endpoints

В коде это будут tRPC-процедуры. Здесь описаны как REST для понятности.

Auth (через Better Auth — в основном автоматически)

```
POST    /auth/sign-up
POST    /auth/sign-in
POST    /auth/sign-out
GET     /auth/session
POST    /auth/verify-email
POST    /auth/forgot-password
POST    /auth/reset-password
```

User

```
GET     /user/me
PATCH  /user/me
GET     /user/settings
PATCH  /user/settings
POST    /user/onboarding/complete
DELETE  /user/me
```

API Keys

```
GET     /keys
PUT     /keys/:provider
DELETE  /keys/:provider
POST    /keys/:provider/validate
```

Workspaces

```
GET     /workspaces
POST    /workspaces
GET     /workspaces/:id
PATCH  /workspaces/:id
DELETE  /workspaces/:id
POST    /workspaces/:id/restore
```

Search

```
POST    /workspaces/:id/search
        body: { query?: string }
        returns: { results: ResourceCandidate[] }
```

Resources

```
POST    /workspaces/:id/resources
GET     /resources/:id
PATCH  /resources/:id
DELETE  /resources/:id
```

Notes

```
GET     /resources/:id/notes
POST    /resources/:id/notes
PATCH  /notes/:id
DELETE  /notes/:id
```

Reader

```
POST    /reader/parse
        body: { url: string }
        returns: { title, content_html, byline, ... }
```

Should-have endpoints (потом)

```
POST    /resources/:id/summarize
GET     /resources/:id/summaries
GET     /search/saved
GET     /workspaces/:id/export
```

5. Дизайн-направление

Эстетика

Минималистично-функциональная система для длительной работы без отвлечений. Гибрид Linear (плотность, точность интерфейса) + Readwise (типографика, читательский комфорт).

Что НЕ берём: геймификация, бейджи, прогресс-бары как в Duolingo, glassmorphism, тяжёлые градиенты, иллюстрации-маскоты, яркие цвета как акценты.

Темы

- **Светлая и тёмная — обе первого класса.** Не «темная как костыль».
- **По умолчанию — system** (следует системной теме устройства).
- **Не чистый белый и не чистый чёрный** — серо-тёплая база, чтобы глаза не уставали при длительном чтении.

Плотность

- **Списки и навигация** (workspace list, поисковые результаты, settings) — плотные, как у Linear
- **Контентные области** (чтение статьи, текст заметки) — просторные, с большими отступами, как у Readwise

Типографика

- **UI:** Geist Sans
- **Чтение длинных текстов** (reader-view статей): Source Serif (через Google Fonts)
- **Моноширинный** (код, таймкоды): Geist Mono
- **Базовый размер:** 15-16px на десктопе, 16-17px для контентных областей

Цветовая палитра

Нейтральная база (серые из Tailwind `zinc` или `neutral`) + один сдержанный акцент.

Светлая тема (предварительно)

Фон: `#fafafa`
Карточки: `#ffffff`
Текст основной: `#0a0a0a`

Текст вторичный: #525252
Границы: #e5e5e5
Акцент: #d97706 (янтарный — временный, может измениться)

Тёмная тема (предварительно)

Фон: #0a0a0a
Карточки: #171717
Текст основной: #fafafa
Текст вторичный: #a3a3a3
Границы: #262626
Акцент: #f59e0b

Семантические цвета — стандартные Tailwind: `red-*` для удаления, `yellow-*` для предупреждений, `green-*` для успеха.

Микро-взаимодействия

- **Длительность:** 150-200ms для большинства переходов, 300ms для модалок
- **Easing:** `ease-out` для появления, `ease-in` для исчезновения
- **Никаких bounces, springs, отскоков**
- **Hover:** тонкая смена фона/границы
- **Focus:** заметный outline для keyboard navigation

Иконки

Lucide (`lucide-react`). Размер 16px для интерфейса, 20-24px для крупных элементов. Один набор, никаких эмодзи и микса.

Базовый набор для ключевых сущностей:

Workspace → BookOpen / Target
Resource (video) → Play / Video
Resource (article) → FileText / BookText
Note → Pencil / Edit3
Search → Search
Settings → Settings / Sliders
API key → Key
Save / Bookmark → Bookmark
Complete → CheckCircle2

Иллюстрации и фото

- **Иллюстрации** — только onboarding (3-5 экранов). Можно использовать unDraw , Open Doodles или заказать кастомные на этапе дизайна.
- **Фото** — не используем. Только thumbnails YouTube из API.

Девять UX-принципов проекта

Это закон проекта. Возвращаемся к ним при каждом сомнительном решении.

1. **Никаких рекомендаций «вам может быть интересно».** Никогда. Ни на каком экране.
2. **Никакого автоплея.** Видео закончилось → пользователь решает, что дальше.
3. **Никакой бесконечной ленты.** Списки имеют конец. Пагинация, не infinite scroll.
4. **Никаких ярких цветовых уведомлений.** Красные точки, badges, счётчики — только функциональные, не для привлечения.
5. **Никаких автоматических пушей про «активность» и «стрики».** Push-уведомления возможны только по явному запросу пользователя (например, «напомни мне об этом workspace через неделю»).
6. **Никаких стриков, бейджей, очков опыта.** Геймификация = механики удержания = противоречие позиционированию.
7. **Понятный и быстрый escape.** В любой момент пользователь может закрыть просмотр и вернуться к списку. Никаких «вы уверены?» каждый раз.
8. **Без dark patterns.** Кнопки удаления одного цвета с подтверждениями. Прозрачные действия.
9. **Принцип закрытой системы.** Внутри workspace пользователь видит только то, что сам туда положил. Приложение никогда не подмешивает «дополнительные предложения», «похожий контент», «возможно, вас заинтересует». Если пользователь хочет добавить — он явно запускает поиск.

6. План реализации

Базовые предположения

- Один разработчик

- 20-30 часов в неделю (вечера + выходные)
- Параллельное изучение нового стека (особенно React)
- Цель — open-source MVP, который не стыдно показать

Фазы

Фаза 0. Подготовка и изучение (2-3 недели)

Цель: заложить фундамент до начала разработки фич.

1. Изучение React (если ещё не знаешь) — `react.dev`, цель — комфортная работа с hooks, lifecycle, TanStack Query
2. Настройка окружения:
 - GitHub-репозиторий
 - Монорепо с `pnpm` и `Turborepo`
 - TypeScript, ESLint, Prettier
 - Базовый Next.js + Tailwind + shadcn/ui
 - Базовый Hono
 - Drizzle с локальным Postgres (Docker Compose)
 - Better Auth, базовая регистрация работает
3. Деплой пустых заготовок на Vercel + Railway, Cloudflare DNS

Deliverable: регистрация работает, frontend и backend задеплоены, домен настроен.

Фаза 1. Workspace и BYO API keys (3-4 недели)

1. Onboarding профиля (4-5 экранов)
2. Settings: профиль, дефолты, тема, API-ключи
3. Шифрование/расшифровка ключей (AES-256-GCM)
4. Endpoint валидации ключей
5. Главный экран — список workspace'ов
6. Создание workspace с фильтрами

7. Workspace-страница (без поиска и ресурсов)

Deliverable: пользователь регистрируется, проходит onboarding, вводит ключи, создаёт пустой workspace.

Фаза 2. Поиск и подборка (3-4 недели)

1. Endpoint `/workspaces/:id/search` :

- Параллельные запросы к YouTube + Brave
- Применение фильтров на этапе запроса
- Унификация результатов

2. Кэш в `search_results_cache` (24 часа)

3. UI карточек результатов

4. Сохранение ресурсов

5. Отображение сохранённых в workspace

6. Кнопка «Обновить подборку»

Deliverable: полный цикл поиска и сохранения работает.

Фаза 3. Просмотр и заметки (4-5 недель)

Самая сложная фаза.

1. Видео-вьюер:

- YouTube IFrame с правильными параметрами
- Перехват `ended` → `stopVideo()`
- Свой UI поверх плеера
- Сохранение прогресса каждые 10 секунд

2. Reader-view:

- Endpoint `/reader/parse` с Mozilla Readability
- Frontend-рендер с Source Serif
- Fallback при ошибке парсинга

3. Заметки:

- TipTap-редактор
- Кнопка «Вставить таймкод»
- Auto-save с debounce

4. **Возврат к ресурсу:** продолжение видео с сохранённого таймкода, скролл к последнему месту в статье

Deliverable: полный цикл «найти → посмотреть → сохранить заметку → вернуться» работает.

Фаза 4. Полировка и подготовка к релизу (2-3 недели)

1. Тёмная тема везде, без багов
2. Адаптивность под планшет и мобильный браузер
3. Loading states, error states, empty states везде
4. Onboarding-улучшения: пропуск, возврат, понятные объяснения
5. Удаление аккаунта (полное стирание)
6. Soft delete с восстановлением
7. Тестирование всех флоу
8. **Landing page** на маршруте / :
 - Описание продукта одной фразой
 - Проблема, решение, как это работает, FAQ
 - Скриншоты или короткое демо
9. **Documentation** на /docs :
 - Как получить API-ключи (по шагам, со скриншотами)
 - FAQ
 - Self-host инструкции (Docker Compose)
10. README репозитория

Deliverable: продукт готов к публичному релизу.

Фаза 5. Релиз и первые пользователи (1-2 недели интенсивно, дальше ongoing)

1. Публикация на GitHub
2. Show HN
3. Reddit: `r/selfhosted`, `r/opensource`, `r/learnprogramming`
4. Twitter/X — короткий thread с демо
5. Реакция на первые отзывы: оперативные багфиксы, улучшение onboarding по обратной связи

Что НЕ делать: добавлять новые большие фичи в первый месяц после релиза.

Сводный таймлайн

Фаза	Длительность	Deliverable
0. Подготовка	2-3 недели	Пустые приложения задеплоены
1. Workspace + ключи	3-4 недели	Регистрация, onboarding, создание workspace
2. Поиск	3-4 недели	Поиск работает, ресурсы сохраняются
3. Просмотр + заметки	4-5 недель	Полный цикл изучения
4. Полировка	2-3 недели	Готов к показу
5. Релиз	1-2 недели	Опубликован, первые пользователи
Итого до публичного MVP	15-21 неделя (~4-5 месяцев)	

После MVP

Правило: **один месяц наблюдения за пользователями → одна крупная фича за раз.**

Приблизительный порядок Should-фич:

1. AI-резюме материала (1-2 недели) — только если пользователи реально просят
2. Поиск по сохранённому (1 неделя)

3. Экспорт workspace в markdown (3-4 дня)
4. Улучшенная визуализация прогресса (1 неделя)

Мобильное приложение — не раньше 6-9 месяцев после MVP. Стек: Expo + React Native + Expo Router. Переиспользует `packages/api-types` , `packages/core` . UI-слой пишется заново под мобильный UX.

7. Риски и принципы работы

Главные риски одиночного проекта

Риск 1. Фича-пирог. Любая идея «а ещё было бы круто...» — это +1-2 недели к релизу. Веди отдельный список «потом», но в MVP не добавляй ничего сверх скоупа.

Риск 2. Перфекционизм в дизайне. Не зависай на 3 недели, выбирая между двумя оттенками. `shadcn/ui` даёт 80% качественного дизайна из коробки. Идеальный дизайн делается на 5-й итерации, не на первой.

Риск 3. Изучение нового вместо разработки. Не «сначала прочитать всю документацию по tRPC». Учись по ходу: возникла задача → изучаешь конкретно её → делаешь.

Риск 4. Разочарование в середине Фазы 3. Это самая длинная фаза. Будет ощущение, что проект бесконечен. Это нормально и проходит.

Риск 5. Релиз и тишина. Большинство open-source проектов получают 50 звёзд и стихают. Подготовься к этому эмоционально. Качество важнее хайпа.

Принципы работы

1. **Документ — единственный источник истины.** Любое отклонение от него — обсуждай явно и обновляй документ.
2. **Скоуп зафиксирован.** Изменение Must have / Should have / Won't have — отдельное решение, не импульс.
3. **Каждая фаза имеет deliverable.** Не двигайся дальше, пока deliverable текущей фазы не готов.
4. **Сроки — ориентир, не дедлайн.** Опережение — приятно. Отставание на месяц — норма для одиночного проекта. Винить себя за отставание — путь к выгоранию.

5. **Возвращайся к девяти UX-принципам при каждом сомнительном решении.** Они защищают позиционирование продукта.
-

Что не зафиксировано в этом документе

Следующие решения откладываются до соответствующих этапов:

- **Окончательная цветовая палитра** — на этапе дизайна в Figma (текущий янтарный — временный)
 - **Конкретные экраны и wireframes** — на этапе дизайна
 - **Точная стоимость токенов и квот** — на этапе подключения API
 - **Стратегия развития сообщества** — после релиза, по обратной связи
 - **Лицензия репозитория** — рекомендуется MIT, но финальное решение перед публикацией
 - **Доменное имя** — `learnspace.app` использовался как условный пример, реальный домен — на твой выбор
-

Этот документ обновляется по мере принятия новых решений. История изменений ведётся через git-историю репозитория.